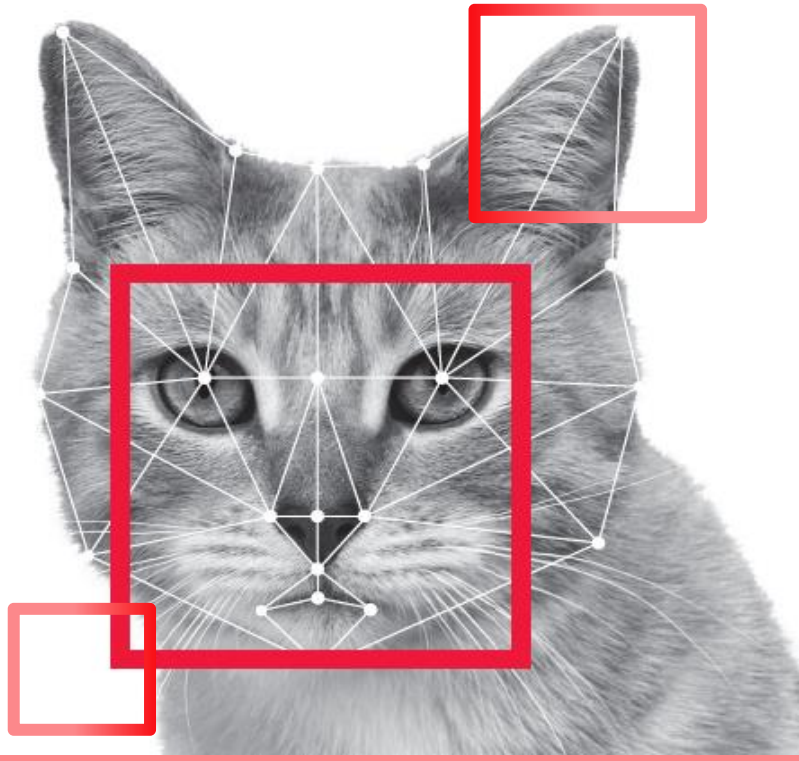
한빛아카데미
Hanbit Academy, Inc.

COMPUTER VISION 컴퓨터 비전

기본 개념부터 최신 모바일 응용 예까지



2장. 영상 처리

각 절에서 다루는 내용

1. 영상 처리의 세 가지 기본 연산

샘플코드에서
sharpen filter
edge
코드 만들어보고
원리를 이해하기

dilate
erosion
팽창→침식
침식→팽창) 효과 비교
iteration

2.4 영상 처리의 세 가지 기본 연산

2.4.1 점 연산

- 오직 자신의 명암값에 따라 새로운 값을 결정

2.4.2 영역 연산

- 이웃 화소의 명암값에 따라 새로운 값 결정

2.4.3 기하 연산

- 일정한 기하 연산으로 결정된 화소의 명암값에 따라 새로운 값 결정



색상

파란 눈 - 빛에 약함
갈색 눈 - 빛에 강함



LCD
LED
OLED (백라이트 X) 소자 자체가 빛

1. 점연산

2. 영역연산
   filtering 개념
   ↓
   Convolution 연산

3. Convolution 연산
   → 신호처리
   ↳ Computer Vision
   Window filter와
   영역과 유사한가
   표현하는 ..

2.4.1 점 연산

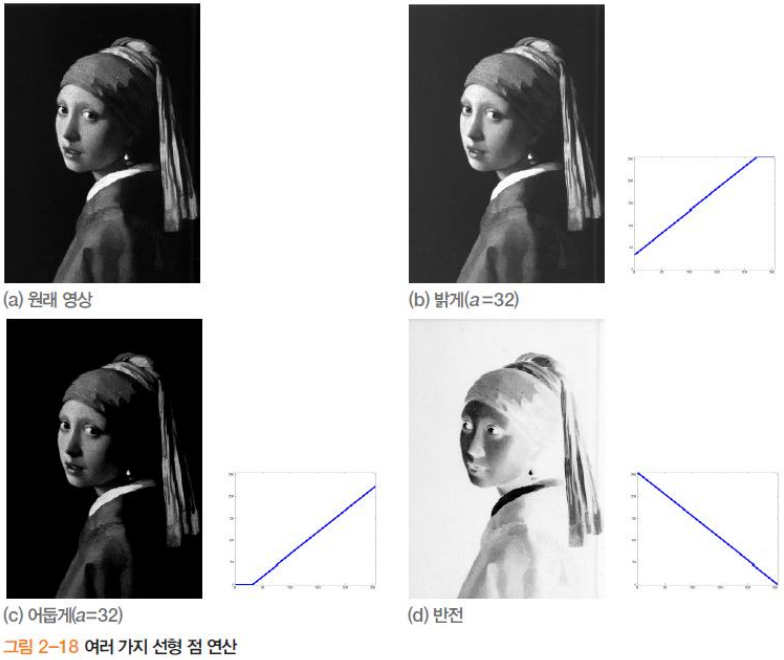
- 점 연산을 식으로 쓰면,
  - 대부분은 k=1 (즉 한 장의 영상을 변환)

$$f_{out}(j,i) = t(f_1(j,i), f_2(j,i), \cdots f_k(j,i)) \qquad (2.10)$$

- 선형 연산
  - 예)

$$f_{out}(j,i) = t(f(j,i))$$

$$= \begin{cases} \min(f(j,i) + a, L-1), & \text{(밝게)} \\ \max(f(j,i) - a, 0), & \text{(어둡게)} \\ (L-1) - f(j,i), & \text{(반전)} \end{cases}$$



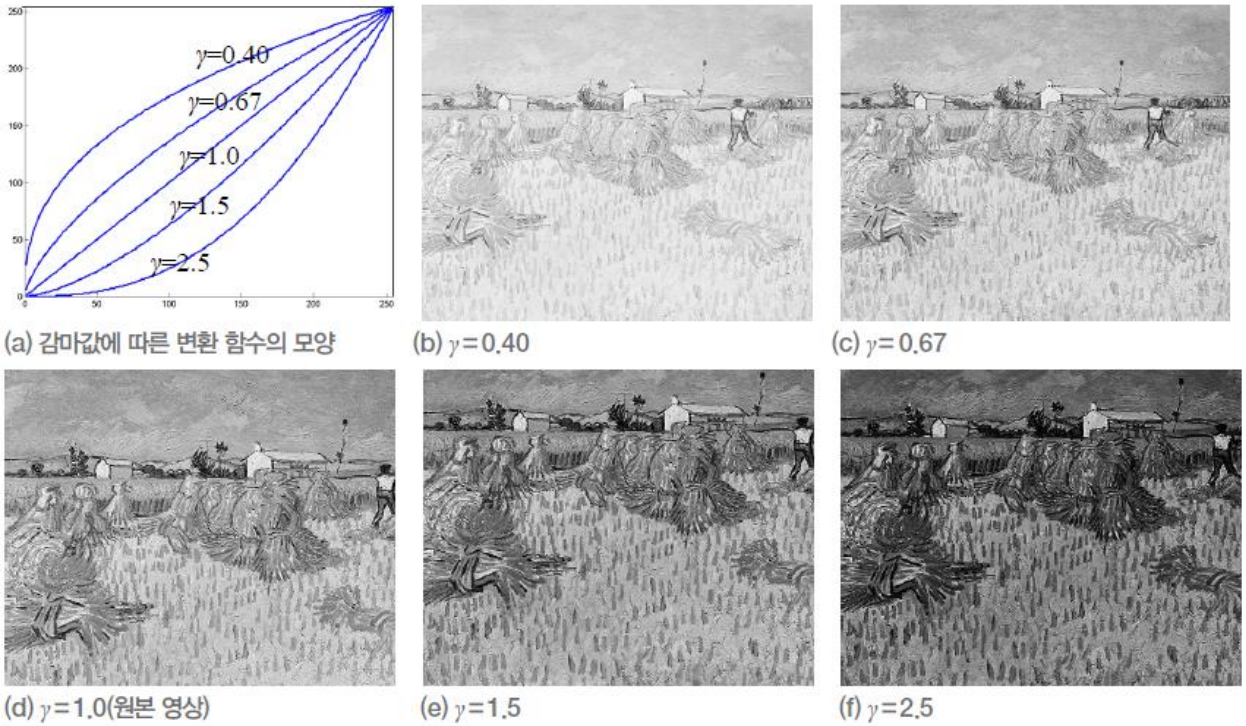
(a) 원래 영상 (b) 밝게(a=32) (c) 어둡게(a=32) (d) 반전

그림 2-18 여러 가지 선형 점 연산

2.4.1 점 연산

- 비선형 연산
  - 예) 감마 수정 (모니터나 프린터 색상 조절에 사용)

$$f_{out}(j,i) = (L-1) \times (\hat{f}(j,i))^{\gamma} \quad \text{이때} \quad \hat{f}(j,i) = \frac{f(j,i)}{(L-1)}$$



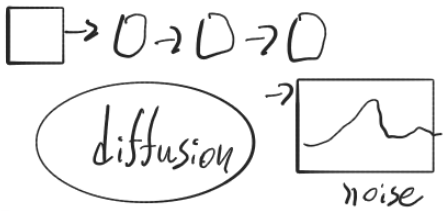
(a) 감마값에 따른 변환 함수의 모양 (b) $\gamma=0.40$ (c) $\gamma=0.67$

(d) $\gamma=1.0$(원본 영상) (e) $\gamma=1.5$ (f) $\gamma=2.5$

그림 2-19 감마 수정

2.4.1 점 연산

- 디졸브 → fade in + fade out
  - k=2인 경우



$$f_{out}(j,i) = \alpha f_1(j,i) + (1-\alpha) f_2(j,i) \qquad (2.13)$$

$0 \le \alpha \le 1$



그림 2-21 디졸브 효과

f_1 + f_2

(j, i) : 화소의 위치

$f(j, i)$: 그 위치의 명암값 $(0 \sim 255)$ $L = 256$

검정 흰색

점 연산 : 자기 자신의 명암값만 사용. 독립적

$$f_out(j, i) = t(f(j, i))$$

t : transformation

- 선형연산 : 밝게 $\min(f + a, L-1)$ (상한선)

  어둡게 $\max(f - a, 0)$ (하한선)

  반전 $(L-1) - f$

  흰색 255 → 0, 검정 0 → 255

- 비선형 연산 : Gamma Correction

  사람 눈과 모니터는 밝기를 비선형적으로 인식

- 디졸브 : fade in + fade out

  $K=2$인 경우 $\Rightarrow \alpha f_1 + (1-\alpha) f_2$ $\quad 0 \le \alpha \le 1$

  연속 영상이 2장

2.4.2 영역 연산 (filtering) ⇒ local feature ⇒ Convolution

- Image filters in spatial domain (영상평면에서의 연산)
  - Filter is a mathematical operation of a grid of numbers
  - Smoothing, sharpening, measuring texture

filtering
→ signal processing

- Image filtering: compute function of local neighborhood at each position

- Really important!
  - Enhance images
    - Denoise, resize, increase contrast, etc.
  - Extract information from images
    - Texture, edges, distinctive points, etc.
  - Detect patterns
    - Template matching

2.4.2 영역 연산

상관 Correlation

- 원시적인 매칭 연산 (물체를 윈도우 형태로 표현하고 물체를 검출)
- 아래 예에서는 최대값 29를 갖는 위치 6에서 물체가 검출됨

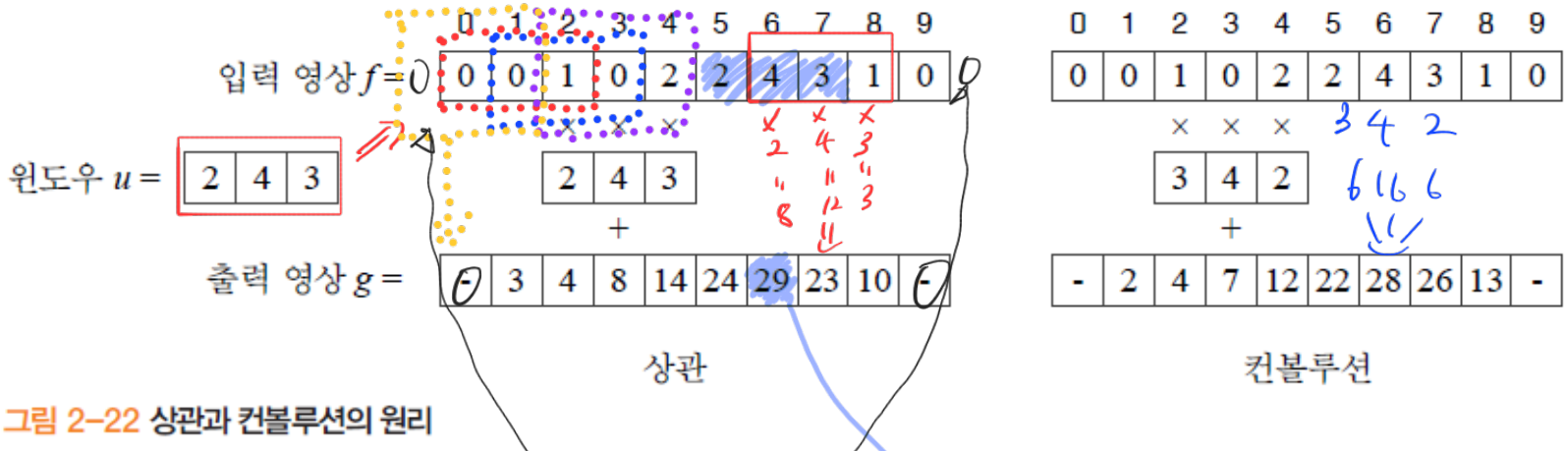


그림 2-22 상관과 컨볼루션의 원리

컨볼루션 Convolution

- 윈도우를 뒤집은 후 상관 적용
- 임펄스 반응

2.4.2 영역 연산

- 2차원

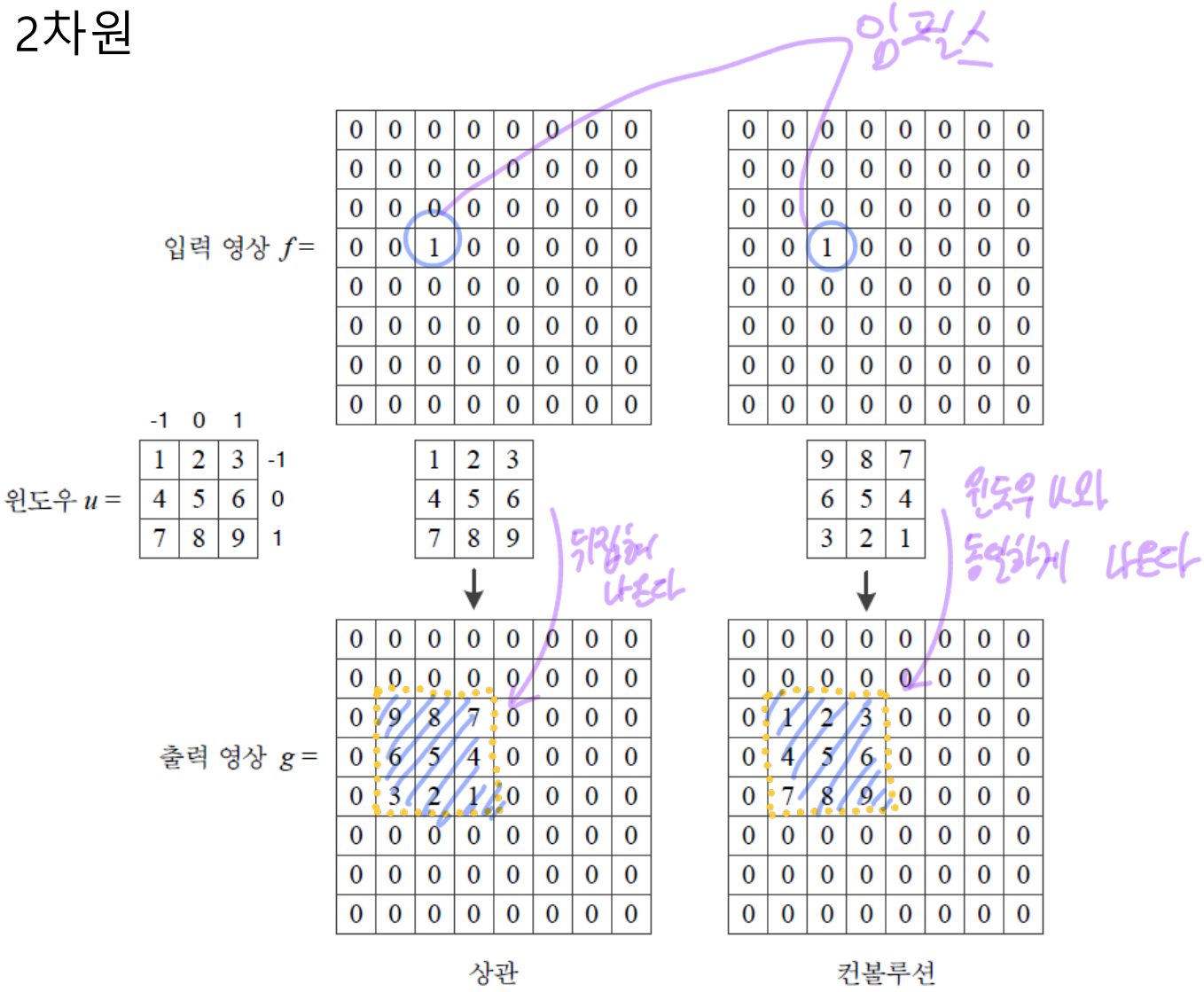




그림 2-23 2차원 상관과 컨볼루션

2.4.2 영역 연산

- 수식으로 쓰면,

$$\left.\begin{aligned} \text{상관 } g(i) = u \otimes f &= \sum_{x=-(w-1)/2}^{(w-1)/2} u(x) f(i+x) \\ \text{컨볼루션 } g(i) = u \circledast f &= \sum_{j=-(w-1)/2}^{(w-1)/2} u(x) f(i-x) \end{aligned}\right\} \text{1차원}$$

$$\left.\begin{aligned} \text{상관 } g(j,i) = u \otimes f &= \sum_{y=-(h-1)/2}^{(h-1)/2} \sum_{x=-(w-1)/2}^{(w-1)/2} u(y,x) f(j+y, i+x) \\ \text{컨볼루션 } g(j,i) = u \circledast f &= \sum_{y=-(h-1)/2}^{(h-1)/2} \sum_{x=-(w-1)/2}^{(w-1)/2} u(y,x) f(j-y, i-x) \end{aligned}\right\} \text{2차원} \qquad (2.14)$$

Convolution 연산

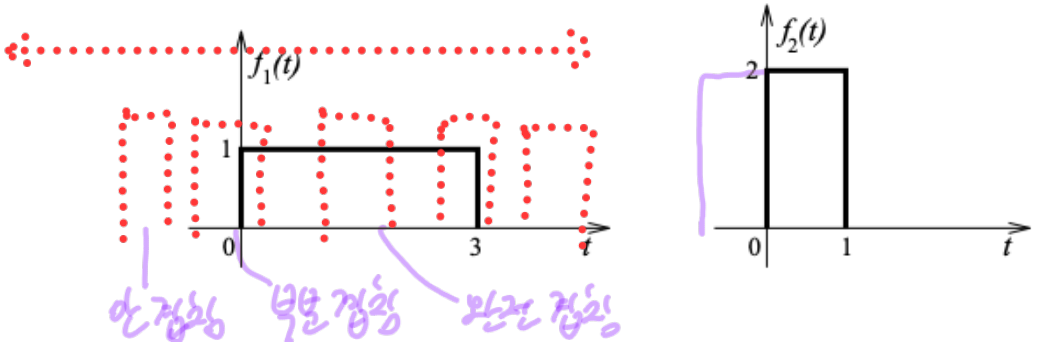
- 이 책은 둘 구분하지 않고 컨볼루션이라는 용어를 사용

Continuous Convolution

- Signal and System 관점에서
  - 두개의 신호 $f_1(t)$ 와 $f_2(t)$ 로 부터 convolution은 아래와 같이 정의

$$f(t) = f_1(t) * f_2(t) = \int_{-\infty}^{\infty} f_1(\tau) f_2(t-\tau) d\tau, \quad -\infty < t < \infty$$

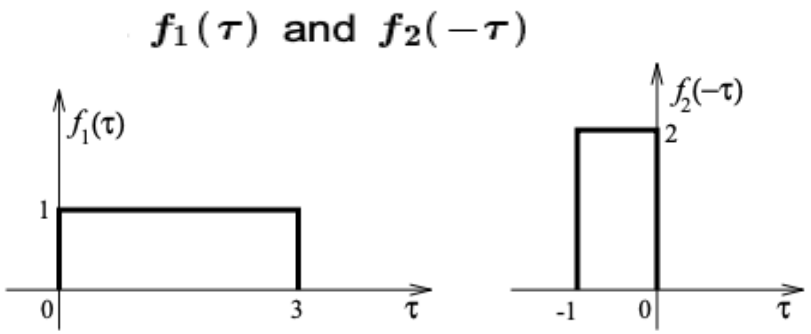
  - 예를 들어 다음과 같이 생각해보기로 함



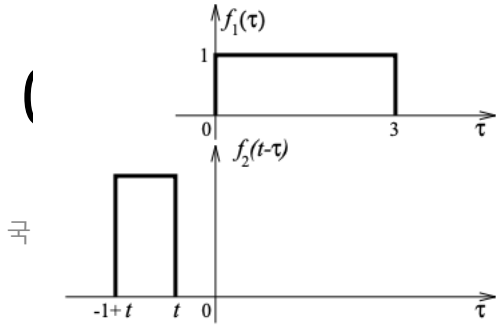
Continuous Convolution

$$f(t) = f_1(t) * f_2(t) = \int_{-\infty}^{\infty} f_1(\tau) f_2(t-\tau) d\tau, \quad -\infty < t < \infty$$

- Step 1



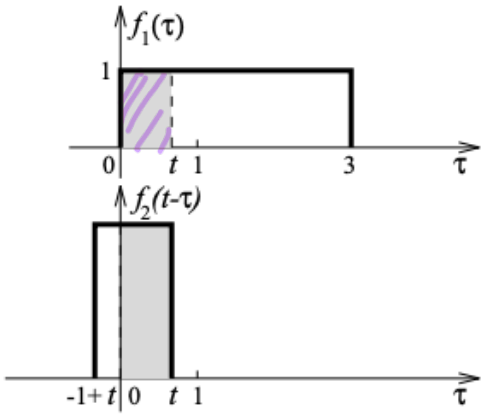
- Step 2: t<0 값은 (



Continuous Convolution

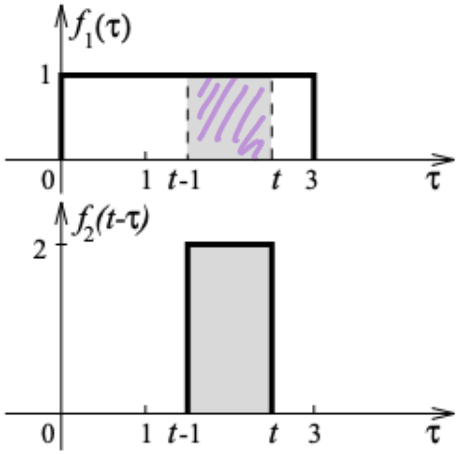
$$f(t) = f_1(t) * f_2(t) = \int_{-\infty}^{\infty} f_1(\tau) f_2(t-\tau) d\tau, \quad -\infty < t < \infty$$

- Step 3 0<t<1



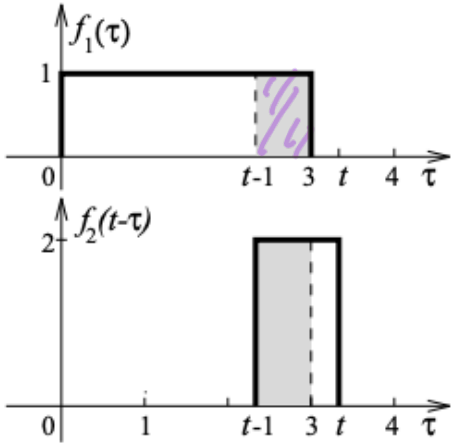
$$f_1(t) * f_2(t) = \int_0^t 1 \times 2 d\tau = 2t, \quad 0 \le t \le 1$$

Step 4 1<t<3



$$f_1(t) * f_2(t) = \int_{t-1}^{t} 1 \times 2 d\tau = 2, \quad 1 \le t \le 3$$

Step 5 3<t<4

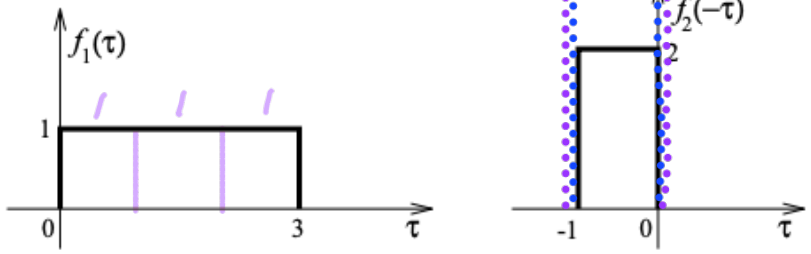


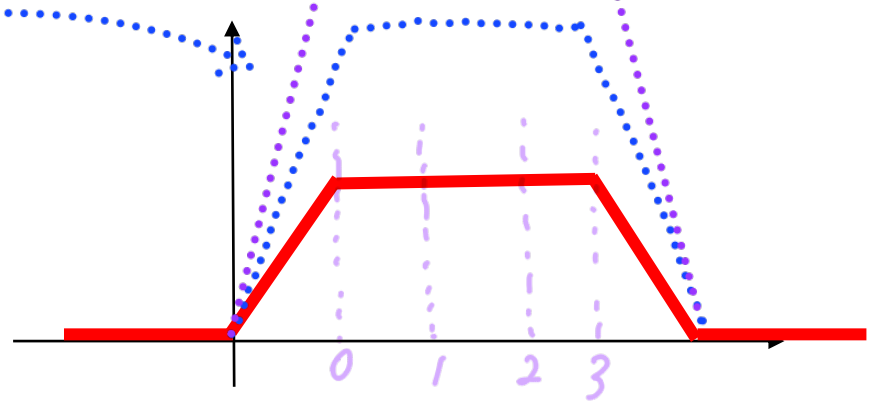
$$f_1(t) * f_2(t) = \int_{t-1}^{3} 1 \times 2 d\tau = 8 - 2t, \quad 3 \le t \le 4$$

Continuous Convolution

mp3 변환 과정

$$f(t) = f_1(t) * f_2(t) = \int_{-\infty}^{\infty} f_1(\tau) f_2(t-\tau) d\tau, \quad -\infty < t < \infty$$





면적 계산

$$f_1(t) * f_2(t) = \begin{cases} 0 & t \le 0 \\ 2t & 0 \le t \le 1 \\ 2 & 1 \le t \le 3 \\ 8-2t & 3 \le t \le 4 \\ 0 & t \ge 4 \end{cases}$$

Discrete Convolution

$$f(t) = f_1(t) * f_2(t) = \int_{-\infty}^{\infty} f_1(\tau) f_2(t-\tau) d\tau, \quad -\infty < t < \infty \quad \Rightarrow \quad y[n] = x[n] * h[n] = \sum_{k=-\infty}^{\infty} x[k] \; h[n-k]$$

x[k] = [3 1 2] h[k] = [3 2 1]

k:	-2	-1	0	1	2	3	4	5
x[k]:			3	1	2			
h[-k]:	1	2	3					
h[1-k]:		1	2	3				
h[2-k]:			1	2	3			
h[3-k]:				1	2	3		
h[4-k]:					1	2	3	
h[5-k]:						1	2	3

y[0] = 3 x 3 = 9

y[1] = 3 x 2 + 3 x 1 = 9

y[2] = 3 x 1 + 1 x 2 + 2 x 3= 11

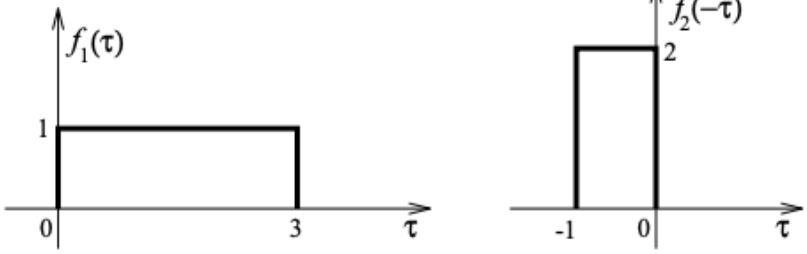
y[3] = 1 x 1 + 2 x 2 = 5

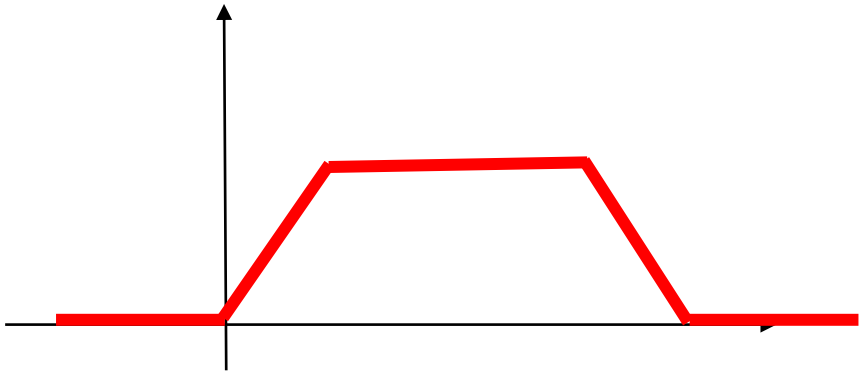
y[4] = 2 x 1 = 2

$y[n] = \{9 \;\; 9 \;\; 11 \;\; 5 \;\; 2 \;\; 0\}$

Discrete Convolution

- 신호 및 시스템에서의 Convolution
  - 미분 가능하지 않은 함수를 미분 가능하게 만들어 줌





  - 다양한 형태의 값을 활용한 필터로 활용
    - 영상처리 및 컴퓨터 비전에서 이를 통해서 특징 강조하기 위한

영역 연산 : 각 화소와 주변 이웃을 함께 계산
(filtering)

필터 = 커널 = 마스크

3x3, 5x5 등등

Correlation (상관) ⊗

: 윈도우 그대로 적용

결과가 뒤집혀 나옴

Matching 연산

윈도우(찾고 싶은 패턴)와

비슷할수록 결과값↑

★중요

Convolution ⊛

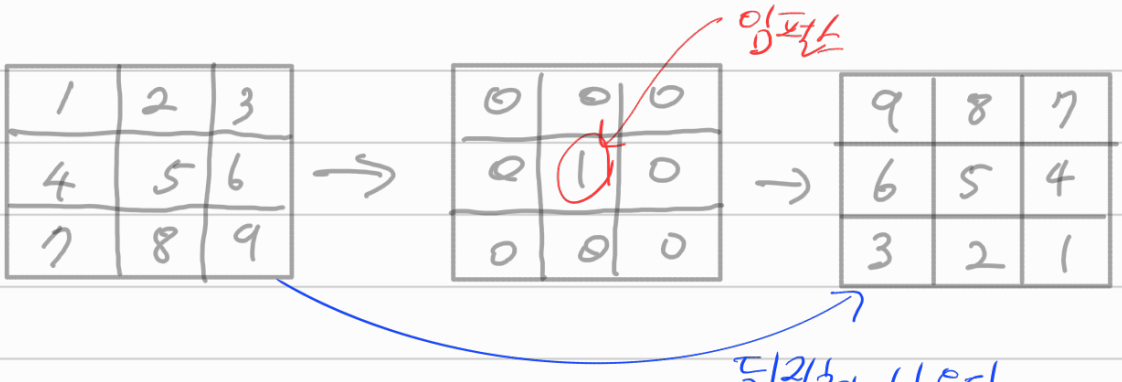
: 윈도우 뒤집은 후 Correlation 적용

윈도우 내와 동일하게 나옴

임펄스를 수학적으로 올바르게 처리하려면

Convolution 사용!

윈도우 w | 2 | 4 | 3 | ⟷ | 3 | 4 | 2 |



9	8	7
6	5	4
3	2	1

⇒ 그래서 처음에 윈도우를 뒤집는다

⇒ Convolution

대칭 필터에서는 상관 = 컨볼루션 ⇒ 그래서 이 책은 둘 구별X

Continuous에서 (연속 신호 : ex) 사각 펄스)

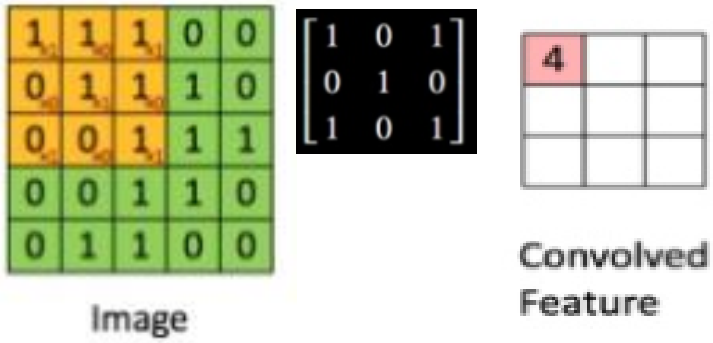
각진 (미분 불가능) 함수도 Convolution 하면 ⇒ 미분 가능

Discrete에서는 (이산 신호 : ex)[9, 9, 11, 5, 2, 0])

미분X but 매끄럽게 해줌

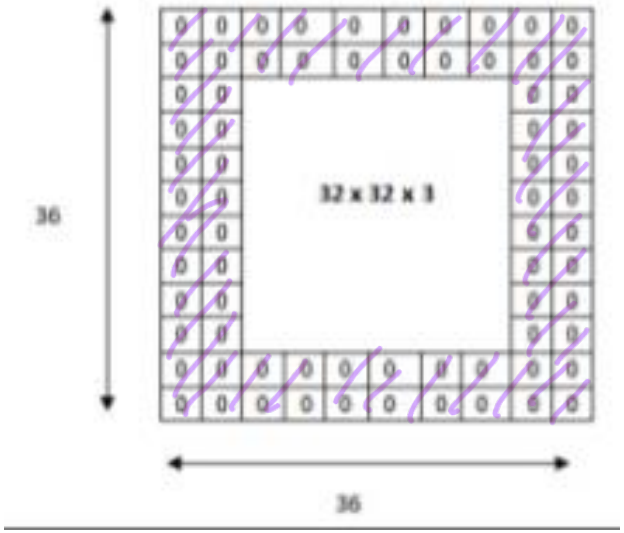
Convolutional Neural Network

- Convolution
  - Local feature를 찾기 위한 과정
  - Hierarchical 구조
  - 필터의 각 셀의 파라미터화
    - 각 layer에서는 여러 필터를 공유
    - Kernel=Filter=Weight=Feature matrix = Feature Map = Activation Map = Parameters to be trained

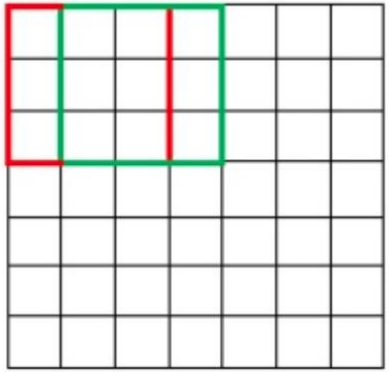


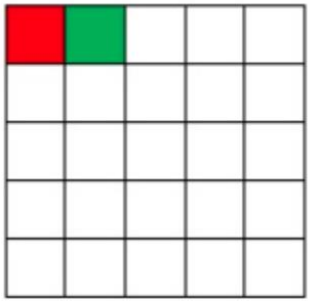
Convolutional Neural Network

- Padding



- Step (stride)





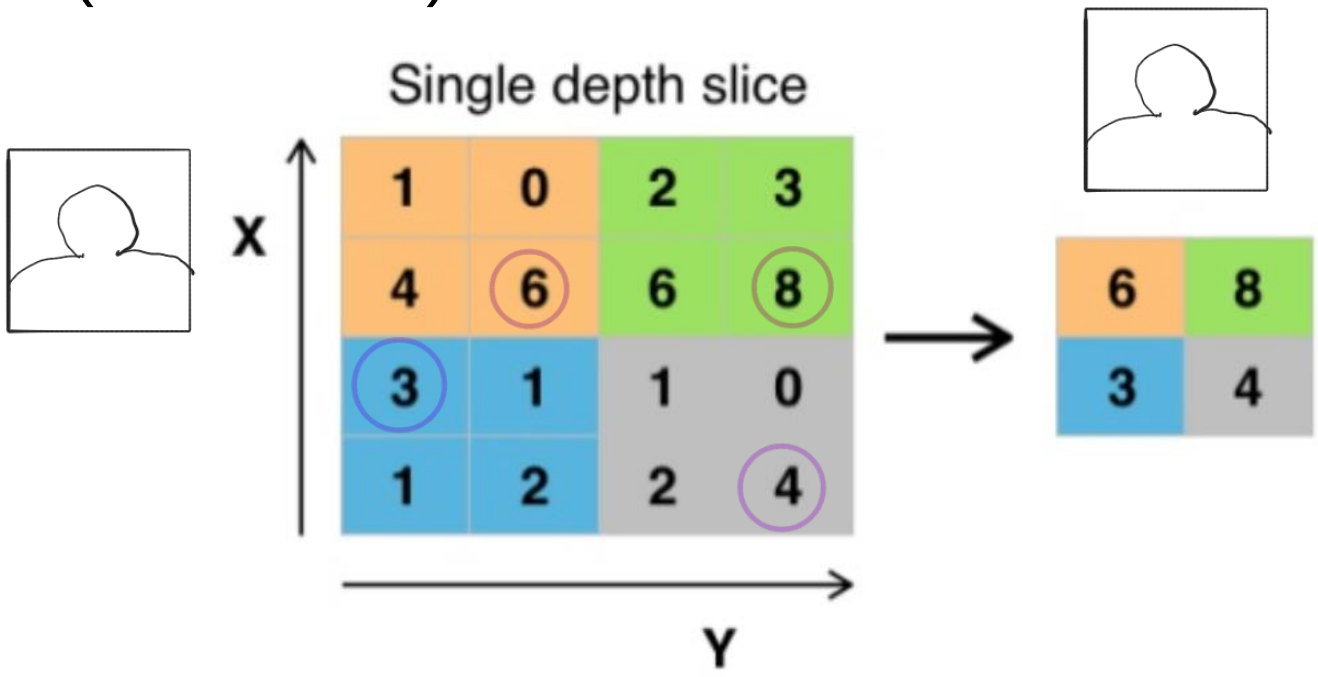
Padding, Stride 설명

Padding의 크기는 입력의 크기에 영향 받는다

또는 Stride(보폭)에 영향 받음

Convolutional Neural Network

- Pooling
  (MaxPool)



Convolutional Neural Network

: Convolution

→ local feature 찾기

계층적 구조

각 Layer에서 여러 필터 공유

kernel = filter = Weight

= parameters to be trained

작은 윈도우(필터)로 가까운 이웃 조합의 패턴

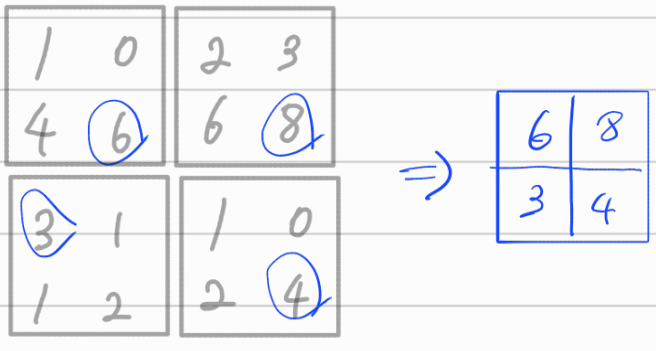
⇒ 의미 있는 패턴 탐지

Padding 크기는 윈도우 크기에 영향

stride는 보폭(stride)에 영향

Pooling : 영역의 대표값 하나 뽑아서 영상 크기 줄이기

└ Maxpool (최댓값) // 평균 사용하면 강한 특징 희석



속도↑, 특징 보존, 위치 변화에 둔감

입력 영상 → [convolution] → Pooling → 반복=Hierarchical → 고차원 판단

t Padding

t Stride

2.4.2 영역 연산

- 컨볼루션 예제
  - 박스와 가우시안은 스무딩 효과
  - 샤프닝은 명암 대비 강조 효과
  - 수평 에지와 수직 에지는 에지 검출 효과
- 컨볼루션은 선형 연산

$$\frac{1}{9}\cdot a_1 + \frac{1}{9}\cdot a_2 + \cdots + \frac{1}{9}\cdot a_9 = \frac{1}{9}\cdot\sum_{i=0}^{9} a_i$$

박스

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

가우시안

.0000	.0000	.0002	.0000	.0000
.0000	.0113	.0837	.0113	.0000
.0002	.0837	.6187	.0837	.0002
.0000	.0113	.0837	.0113	.0000
.0000	.0000	.0002	.0000	.0000

샤프닝

0	-1	0
-1	5	-1
0	-1	0

수평 에지

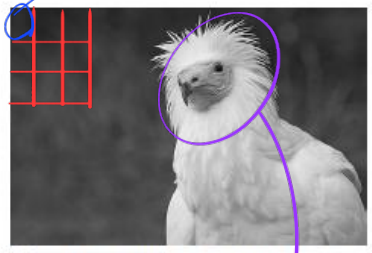
1	1	1
0	0	0
-1	-1	-1

수직 에지

1	0	-1
1	0	-1
1	0	-1

모션

.0304	.0501	0	0	0
.0501	.1771	.0519	0	0
0	.0519	.1771	.0519	0
0	0	.0519	.1771	.0501
0	0	0	.0501	.0304



(a) 원래 영상과 여러 가지 마스크들



> 박스



> 가우시안



> 샤프닝



> 수평 에지



> 수직 에지



> 모션

(b) 다양한 마스크로 컨볼루션한 영상들

그림 2-24 다양한 마스크와 컨볼루션 효과

Example: box filter

$g[\cdot,\cdot]$

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Slide credit: David Lowe (UBC)

Image filtering

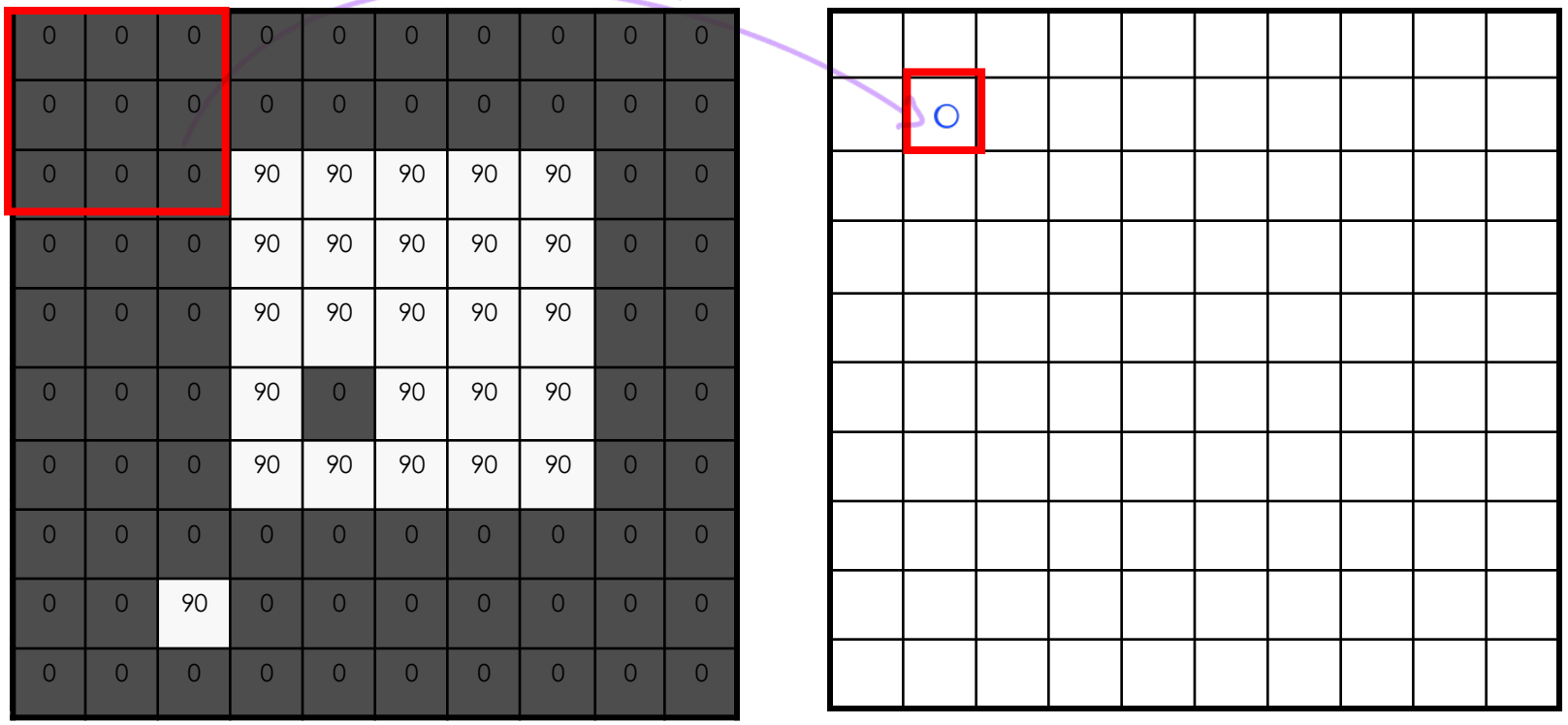
계산과정 시험출제

$g[\cdot,\cdot]$ $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$f[.,.]$ $h[.,.]$

$\frac{1}{9}\times 0+\frac{1}{9}\times 0+\cdots+\frac{1}{9}\times 0 = 0$



$$h[m,n]=\sum_{k,l} g[k,l]\, f[m+k,n+l]$$

Credit: S. Seitz

Image filtering

$g[\cdot,\cdot]$ $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$f[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[.,.]$

	0	10							

$$h[m,n] = \sum_{k,l} g[k,l]\, f[m+k, n+l]$$

Credit: S. Seitz

Image filtering

$g[\cdot,\cdot]$ $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$f[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[.,.]$

	0	10	20						

$$h[m,n] = \sum_{k,l} g[k,l]\, f[m+k, n+l]$$

Credit: S. Seitz

Image filtering

$g[\cdot,\cdot]$ $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$f[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[.,.]$

	0	10	20	30					

$$h[m,n] = \sum_{k,l} g[k,l]\, f[m+k, n+l]$$

Credit: S. Seitz

Image filtering

$g[\cdot,\cdot]$ $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$f[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[.,.]$

	0	10	20	30	30				

$$h[m,n] = \sum_{k,l} g[k,l]\, f[m+k, n+l]$$

Credit: S. Seitz

Image filtering

$g[\cdot,\cdot]$ $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$f[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[.,.]$

	0	10	20	30	30				
				?					

$$h[m,n] = \sum_{k,l} g[k,l]\, f[m+k, n+l]$$

Credit: S. Seitz

Image filtering

$g[\cdot,\cdot]$ $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$f[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[.,.]$

	0	10	20	30	30				
						?			
				50					

$$h[m,n] = \sum_{k,l} g[k,l]\, f[m+k, n+l]$$

Credit: S. Seitz

Image filtering

$g[\cdot,\cdot]$ $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

① Smooth

$f[.,.]$ → $h[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

②

②noise(out layer) 줄여준다

Salt & Pepper noise

$$h[m,n] = \sum_{k,l} g[k,l]\, f[m+k, n+l]$$

Credit: S. Seitz

Box Filter

What does it do?

- Replaces each pixel with an average of its neighborhood

- Achieve smoothing effect (remove sharp features)

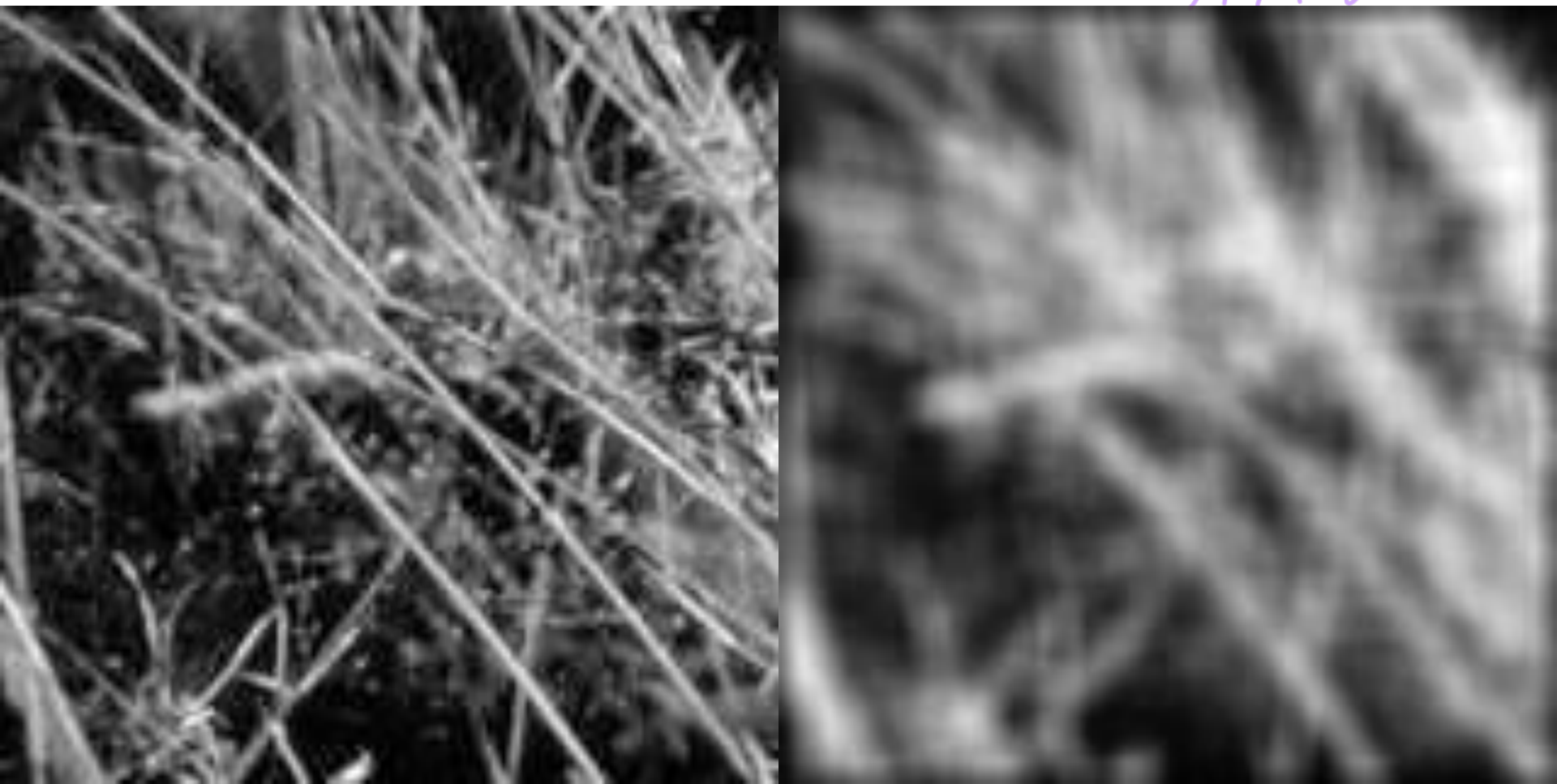
$g[\cdot,\cdot]$

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Slide credit: David Lowe (UBC)

Smoothing with box filter

Mosaic



Practice with linear filters



Original

*

0	0	0
0	1	0
0	0	0

→ 그대로 내용

?

Source: D. Lowe

Practice with linear filters



Original

0	0	0
0	1	0
0	0	0



Filtered
(no change)

Source: D. Lowe

Practice with linear filters



Original

0	0	0
0	0	1
0	0	0

?

오른쪽 픽셀이 내 픽셀이 된다

→ 왼쪽으로 Shift 1만

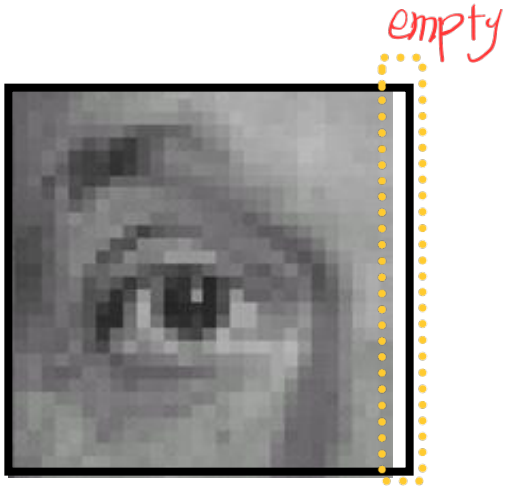
Source: D. Lowe

Practice with linear filters



Original

0	0	0
0	0	1
0	0	0



Shifted left
By 1 pixel

Source: D. Lowe

Practice with linear filters



Original

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

box filter

(Note that filter sums to 1)

?

$$\begin{bmatrix} -\frac{1}{9} & -\frac{1}{9} & -\frac{1}{9} \\ -\frac{1}{9} & \frac{17}{9} & -\frac{1}{9} \\ -\frac{1}{9} & -\frac{1}{9} & -\frac{1}{9} \end{bmatrix}$$

$$\square * \text{filter}_1 \Rightarrow \tilde{\square} * \text{filter}_2$$

$$\square * (f_1 - f_2)$$

Source: D. Lowe

Practice with linear filters



Original

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$



Sharpening filter

- Accentuates differences with local average

Source: D. Lowe

box filter (영상 smooth하게, sharp 없앰)
경계 없이 생성됨, blur

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

$\Rightarrow \frac{1}{9}a_1 + \frac{1}{9}a_2 + \cdots + \frac{1}{9}a_9$

noise (90, 0) 값↑ ⇒ 60, 10으로 noise 줄여준다

0	0	90
0	90	0
90	0	0

⇒

10	20	20
20	30	20
20	20	10

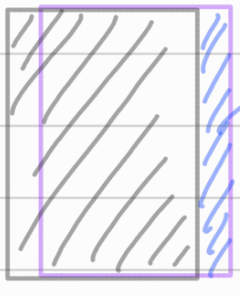
: 완만해진다

---

original × | 0 0 0 / 0 1 0 / 0 0 0 | ⇒ original

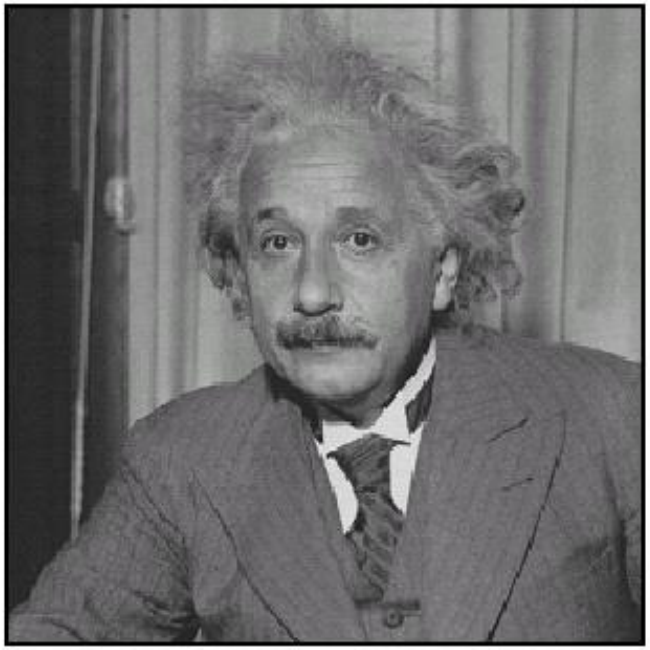
변화 X

× | 0 0 0 / 0 0 1 / 0 0 0 | ⇒

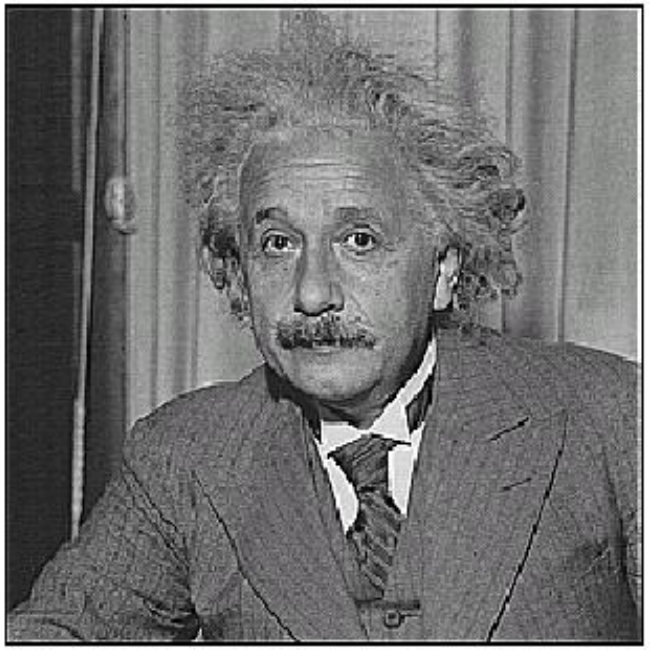


왼쪽으로 shift 1번

Sharpening



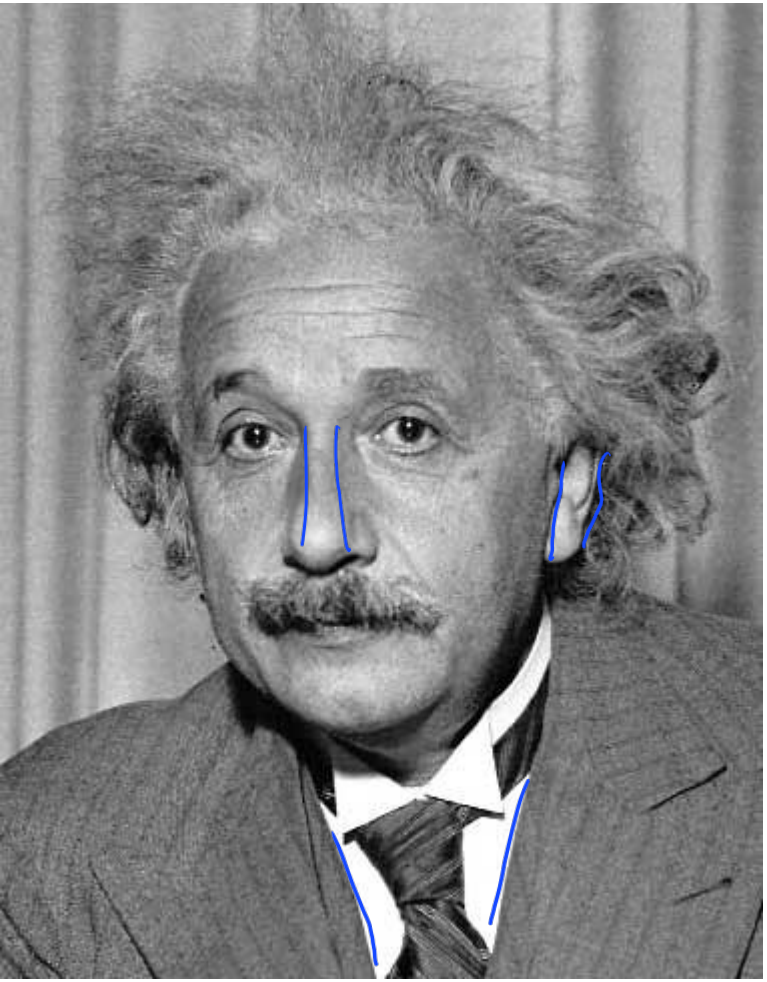
before



after

Source: D. Lowe

Other filters

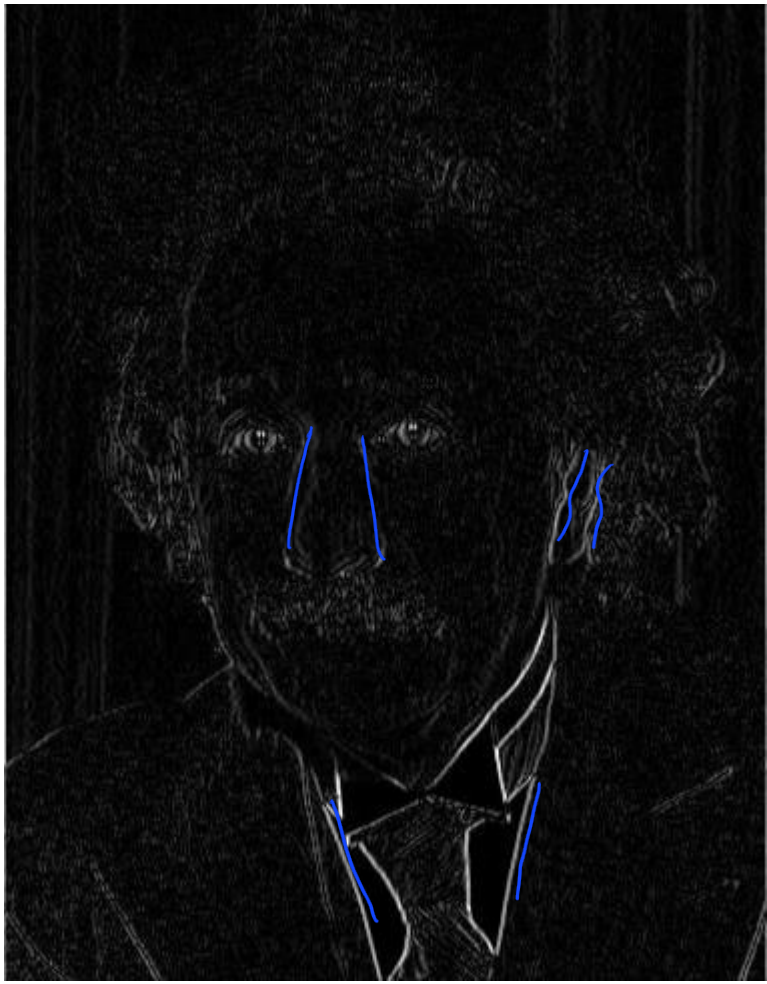


Prewitt

1	0	-1
1	0	-1
1	0	-1

1	0	-1
2	0	-2
1	0	-1

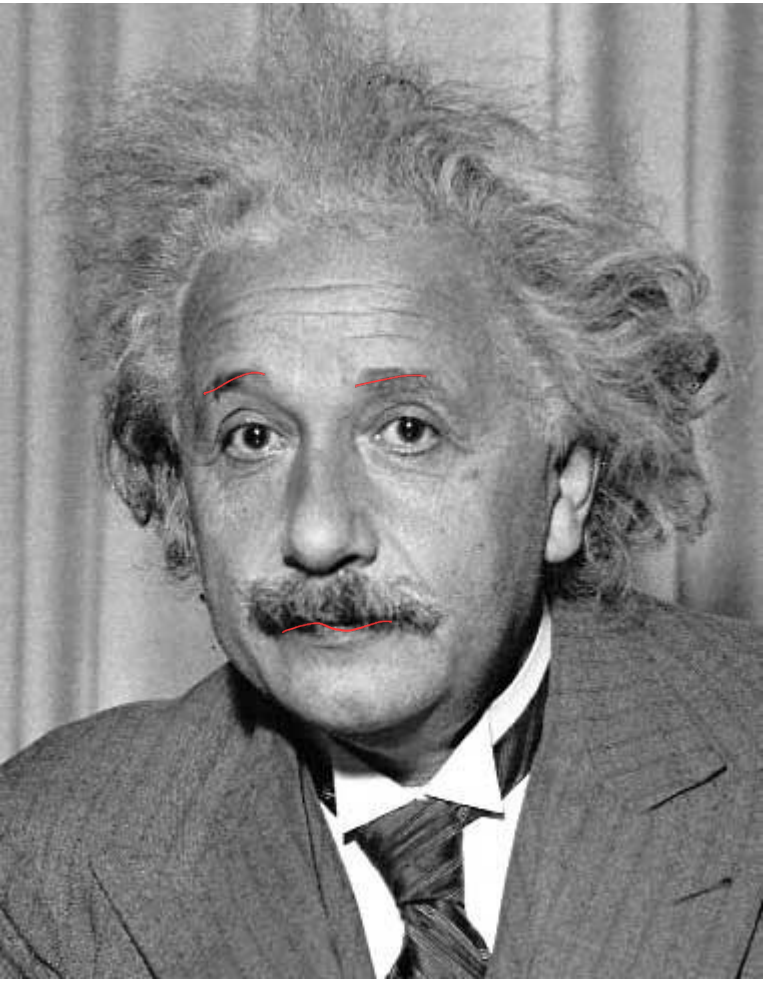
Sobel



Vertical Edge
(absolute value)

차선 인식 - 자율주행

Other filters



1	2	1
0	0	0
-1	-2	-1

Sobel



Horizontal Edge
(absolute value)

Key properties of linear filters

Linearity:

$\frac{f_{11}+\cdots+f_{19}}{9} + \frac{f_{21}+\cdots+f_{29}}{9} = \frac{1}{9}(f_{11}+\cdots+f_{19}+f_{21}+\cdots+f_{29})$

`filter(f1 + f2) = filter(f1) + filter(f2)`

Shift invariance: same behavior regardless of pixel location

`filter(shift(f)) = shift(filter(f))`

Any linear, shift-invariant operator can be r epresented as a convolution

Source: S. Lazebnik

More properties

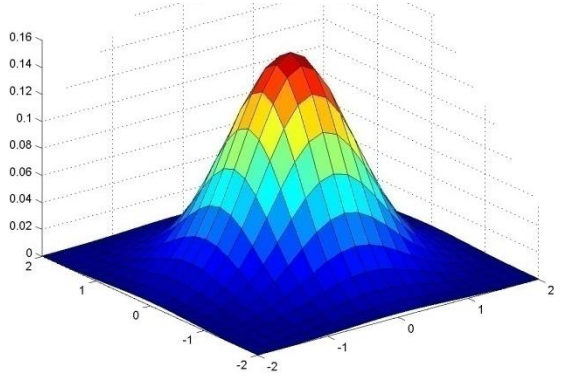
- Commutative: $a * b = b * a$
  - Conceptually no difference between filter and signal
  - But particular filtering implementations might break this equal ity
- Associative: $a * (b * c) = (a * b) * c$
  - Often apply several filters one after another: $(((a * b_1) * b_2) * b_3)$
  - This is equivalent to applying one filter: a * $(b_1 * b_2 * b_3)$
- Distributes over addition: $a * (b + c) = (a * b) + (a * c)$
- Scalars factor out: $ka * b = a * kb = k (a * b)$
- Identity: unit impulse $e = [0, 0, 1, 0, 0]$, $a * e = a$

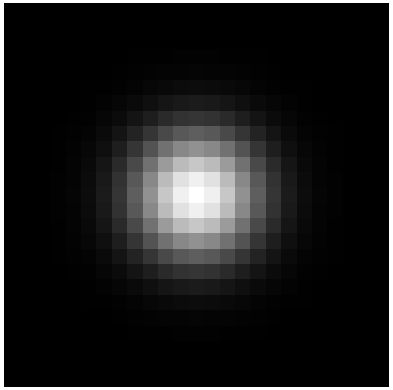
Source: S. Lazebnik

Important filter: Gaussian

가까운 픽셀에 가중치를 준다

- Weight contributions of neighboring pixels by nearness

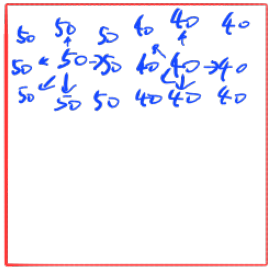




0.003	0.013	0.022	0.013	0.003
0.013	0.059	0.097	0.059	0.013
0.022	0.097	0.159	0.097	0.022
0.013	0.059	0.097	0.059	0.013
0.003	0.013	0.022	0.013	0.003

5 x 5, $\sigma = 1$

$$G_\sigma = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$



모서리

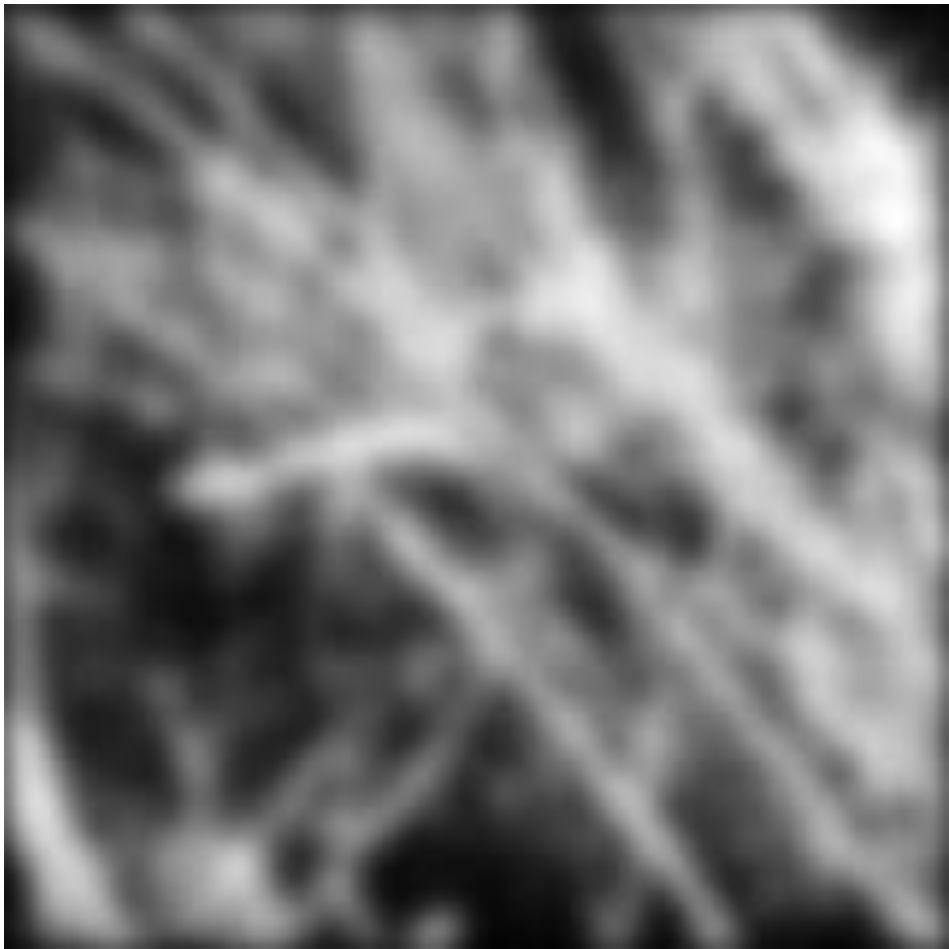
Slide credit: Christopher Rasmussen

Smoothing with Gaussian filter

box filter와 비교했을 때 계단 현상 줄었다



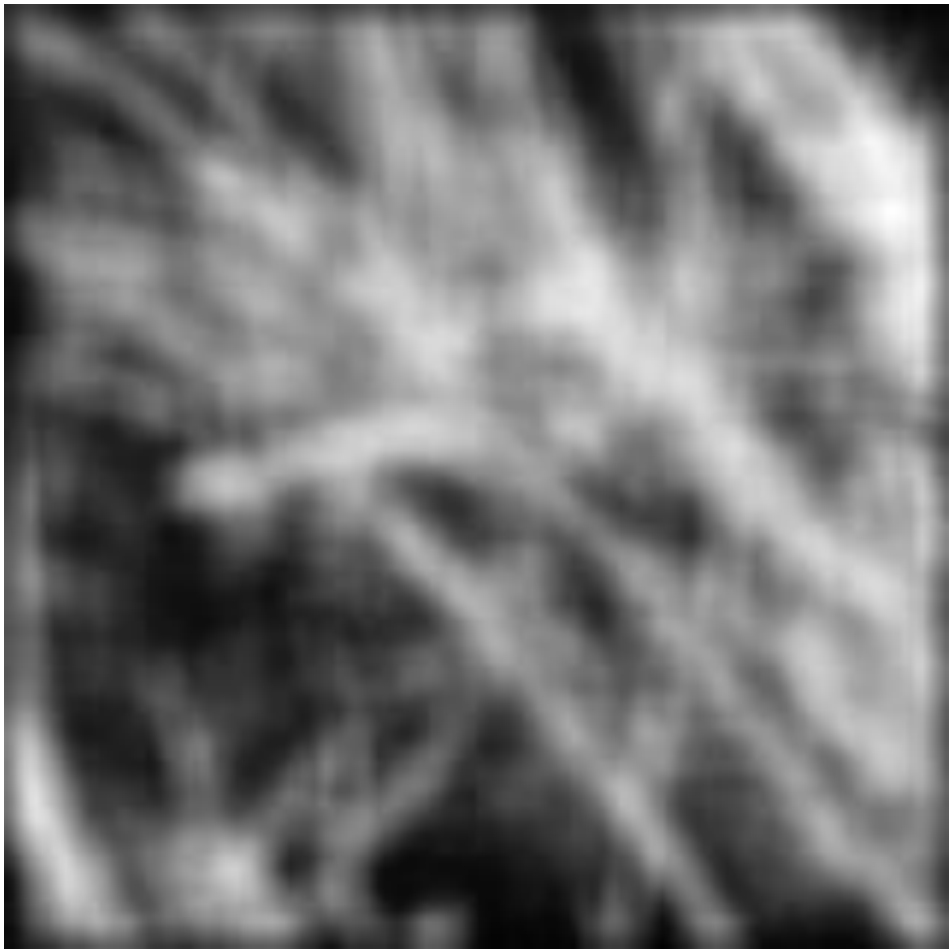




Smoothing with box filter







Gaussian filters

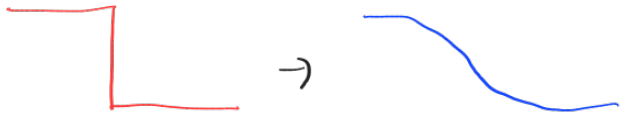


시간의 영역

- Remove "high-frequency" components from the image (low-pass filter)
  - Images become more smooth

명암값의 변화가 큰 곳?
→ High-Frequency

- Convolution with self is another Gaussian
  - So can smooth with small-width kernel, repeat, and get same result as larger-width kernel woul d have
  - Convolving two times with Gaussian kernel of w idth σ is same as convolving once with kernel of width $\sigma\sqrt{2}$
- *Separable* kernel
  - Factors into product of two 1D Gaussians



2D → 1D, 1D

Source: K. Grauman

Separability of the Gaussian filter

$$G_\sigma(x,y) = \frac{1}{2\pi\sigma^2}\exp^{-\frac{x^2+y^2}{2\sigma^2}}$$

$$= \left(\frac{1}{\sqrt{2\pi}\sigma}\exp^{-\frac{x^2}{2\sigma^2}}\right)\left(\frac{1}{\sqrt{2\pi}\sigma}\exp^{-\frac{y^2}{2\sigma^2}}\right)$$

x축 방향 y축 방향

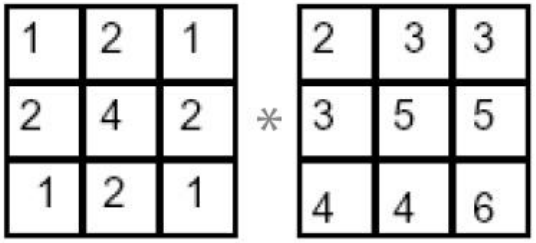
The 2D Gaussian can be expressed as the product of two functions, one a function of x and the other a function of y

In this case, the two functions are the (identical) 1D Gaussian

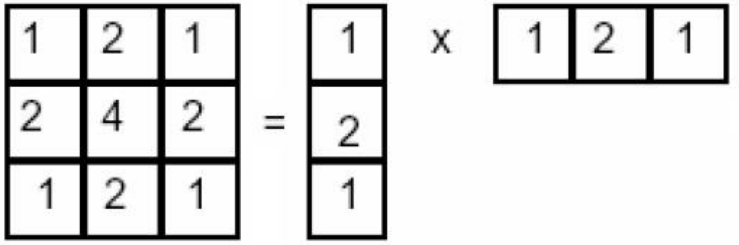
Source: D. Lowe

Separability example

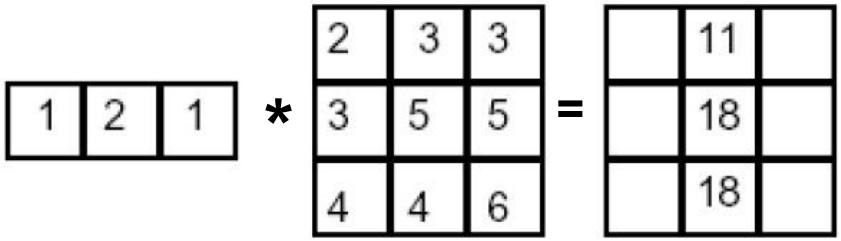
2D convolution
(center location only)



The filter factors
into a product of 1D
filters:



Perform convolution
along rows:



Followed by convolution
along the remaining column:

Source: K. Grauman

Separability

- Why is separability useful in practice?

2.4.2 영역 연산

- 비선형 연산
  - 예) 메디안 필터
    - 솔트페퍼 잡음에 효과적임
    - 메디안은 가우시안에 비해 에지 보존 효과 뛰어남



(a) 원래 영상



(b) 솔트페퍼 잡음

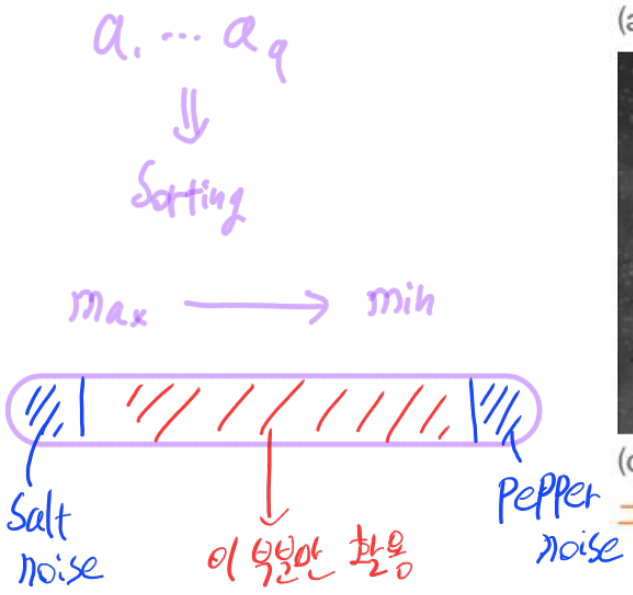


(c) 가우시안 필터



(d) 메디안 필터

그림 2-25 가우시안과 메디안 필터의 비교



Median filters

- A **Median Filter** operates over a window by selecting the median intensity in the window.
- What advantage does a median filter have over a mean filter?
- Is a median filter a kind of convolution?



① box sharpening gaussian

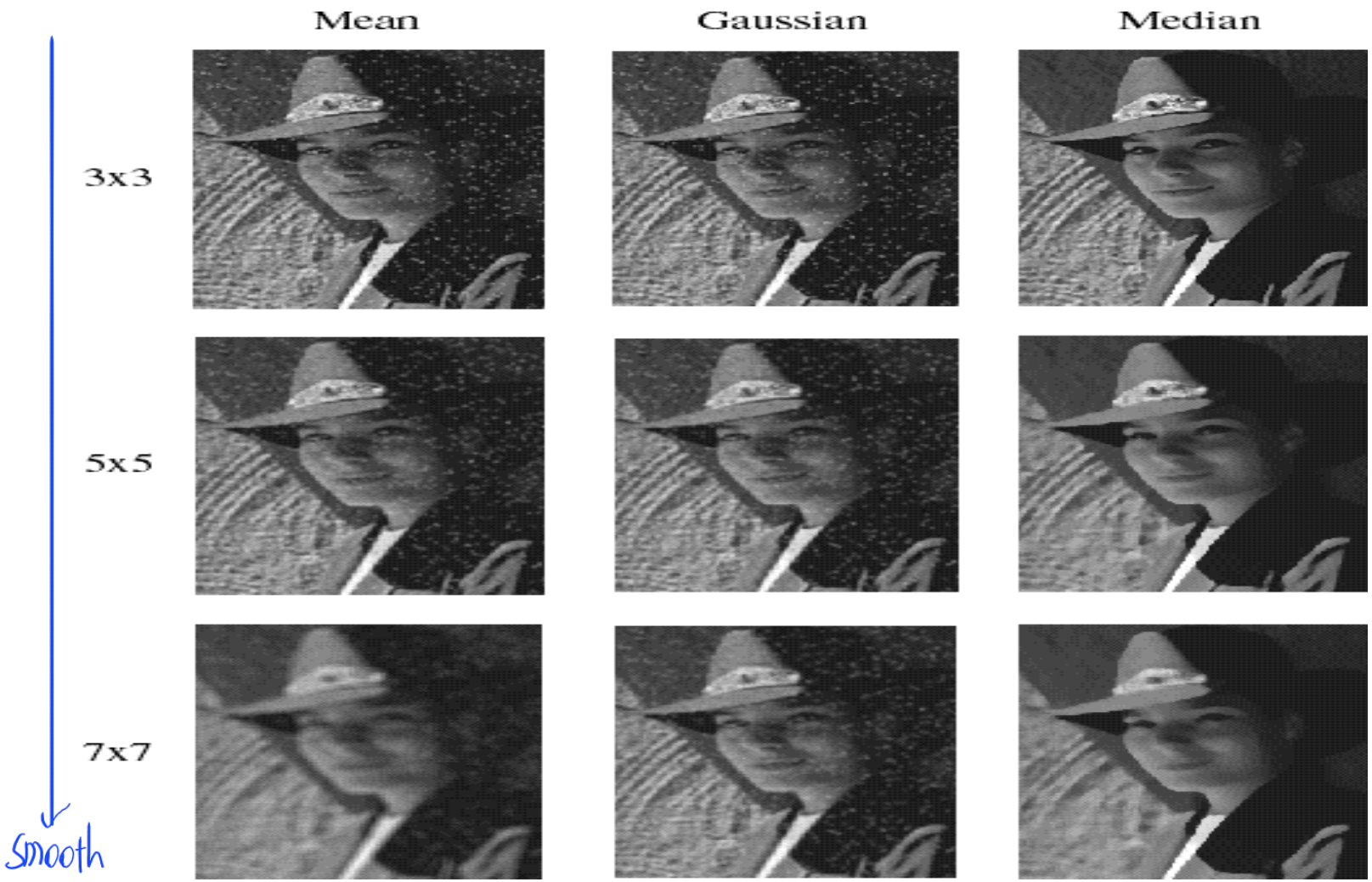
② linear non-linear

③ image 특징 에 맞는 필터 고르기

© 2006 Steve Marschner • 53

Slide by Steve Seitz

Comparison: salt and pepper noise



Slide by Steve Seitz